DOCUMENTACIÓN TÉCNICA: Cómo Ofuscar y Minificar una Página Web Profesional.

Inicio y fin: 23/3/26 – 24/3/26.

Título del Proyecto: Ofuscación y Despliegue de Alta Seguridad

Herramientas: Vite (Build Tool), Terser (Minificador), Firebase (Hosting).

Estado Final Deseado: Código **HTML**/**CSS**/**JS** comprimido en una sola línea ~~INFINITA~~, con lógica incomprensible y encriptación de textos.

ÍNDICE.

PRERREQUISITOS: Arquitectura y Modularización.

1. **PASO 1: Estructura de Carpetas y Configuración de Hosting (Firebase).**
2. **PASO 2: Instalación del Entorno de Construcción (Node.js).**
3. **PASO 3: Evolución de la Configuración de Vite (vite.config.js).**
4. **PASO 4: El Proceso de Construcción (The Build).**
5. **PASO 5: *Despliegue y Verificación Final.***

---

1. PRERREQUISITOS: Arquitectura y Modularización.

Antes de iniciar cualquier proceso de ofuscación, es **obligatorio** que el proyecto esté modularizado. El ofuscador no funciona correctamente si el HTML tiene JavaScript embebido (dentro de etiquetas <script>) en un Enfasis que quiero realizar en este punto quiero aclarar que no es que no se pueda tener absolutamente nada de java script en el documento HTML, pero el nivel de JS que se puede tener en el documento html es mínimo muy mínimo para evitar que en la compresión de texto por la herramienta (Vite) sea tan fuerte que según el nivel de preferencia que desees tener este no afecte la funcionalidad de los documentos

que se entrelazan para dar como finalidad una función de forma 100% optima sin fallar en absolutamente nada asi que (OJO) con eso.

A Continuación, te mostrare algunos ejemplos:

puntalmente ambiguos de que tan permitido se puede tener una pequeña cantidad de JavaScript embebido (dentro de etiquetas <script>) en el **HTML.**

```
<script type="text/javascript">
    function googleTranslateElementInit() {
        new google.translate.TranslateElement({
            pageLanguage: 'es',
            includedLanguages: 'es,en,fr,pt,ht,ru,zh-CN',
            layout: google.translate.TranslateElement.InlineLayout.SIMPLE,
            autoDisplay: false
        }, 'google_translate_element');
    }
</script>
```

Y este ejemplo que ves aquí arriba es lo máximo a nivel de scripts por documento **HTML** que te puedes permitir tener tan siquiera antes de pensar en ofuscar y Minificar todo tu proyecto.

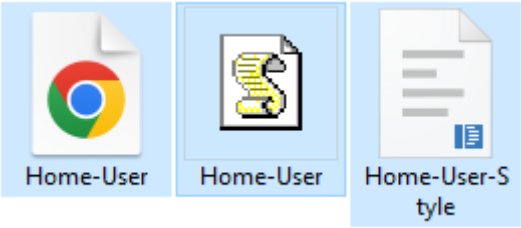
Regla de Oro: 1 Documento Raíz (HTML) = 1 Hoja de Estilos (CSS) + 1 Hoja de Funciones Motrices (JS). Esta estructura de jerarquías de documentos es 100% **OBLIGATORIA PARA EL CORRECTO FUNCIONAMIENTO**, de todos tus documentos entrelazados luego de separarlos o haber creado tu proyecto desde cero en este orden y si no esta asi cuando vallas modularizando cada parte del proyecto porque tienes las jerarquías mezcladas tipo un **HTML CON JS EMBEBIDO DENTRO DEL MISMO HTML Y UN DOCUEMOTO CSS POR FUERA ENTRELAZADO CON CONECXIÓN EXTERNA,** al momento de ir modularizando cada documento e independizarlo debes tomar en cuenta que todo funcione cuando ya todo este modularizado porque de lo contrario te será casi imposible encontrar los errores si ofuscas el codigo de todos los archivos que conforman tu proyecto un ejemplo de cómo debe quedar toda tu Modularización es de la siguiente manera:

Estado Requerido del Proyecto:

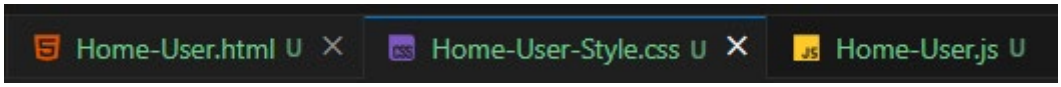
Cada página debe estar separada en tres archivos con el mismo nombre para mantener el orden:

- **login.html**
- **login.js**
- **login.css** (Vinculado en el HTML como <script type="module" src="login.js"></script>).

Ejemplo 1 en Carpeta:



Ejemplo 2 en el Editor: (Que estes Utilizando en este Momento**):**



---

2. PASO 1: Estructura de Carpetas y Configuración de Hosting (Firebase)

En este proyecto, utilizamos Firebase como motor de Hosting. Es crucial configurar la estructura de carpetas correcta para que Vite y Firebase no choquen.

El Rol de la Carpeta `public`

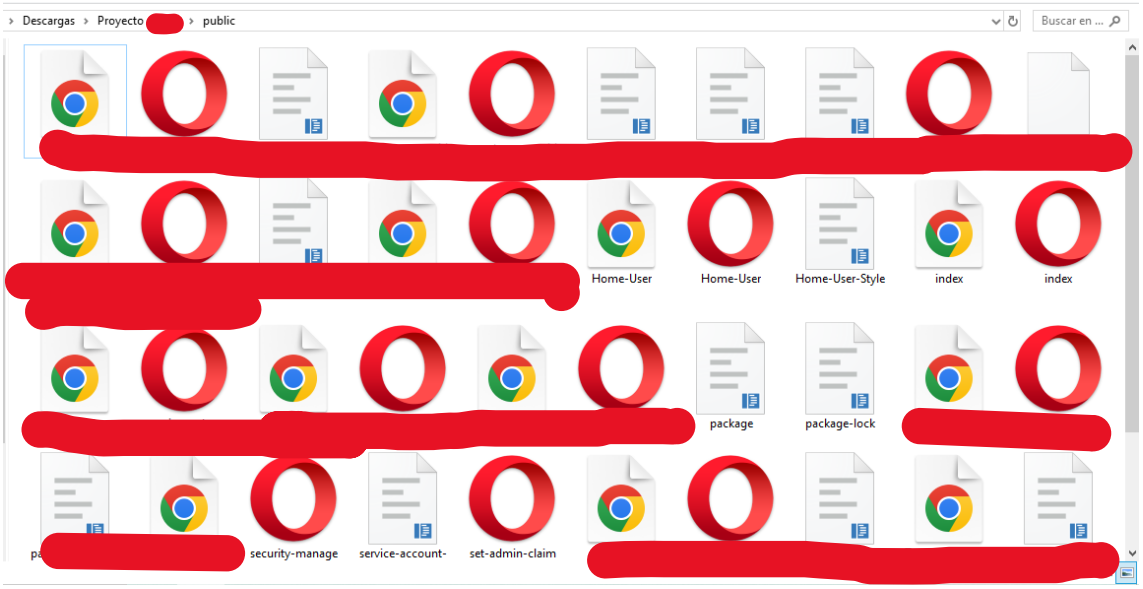
Vite utiliza una carpeta especial llamada `public`. Para este ejemplo Genérico e Ilustrativo decidimos llamar a nuestra carpeta donde tenemos todo el proyecto (public) Todo lo que esté adentro se copiará a la carpeta final de producción (`dist`) **sin ser tocado, renombrado ni comprimido** por *nosotros*. ATENCION! A lo que se esta refiriendo este texto de ilustración funcional sobre cómo funciona la herramienta (Vite) este texto quiere decir que (*todo el proyecto (public) Todo lo que esté adentro se copiará a la carpeta final de producción (`dist`)* ***sin ser tocado, renombrado ni comprimido***.)

lo que esto quiere decir es que la herramienta (Vite) en la terminal cuando se ejecuta porque ya fue configurada esta hace una copia de toda tu carpeta Principal que es donde Tenías todo tu proyecto inicialmente. A qué me refiero, por ejemplo.?

los proyectos cuando se están creando los proyectos webs primero se crea una carpeta raíz que es la primera carpeta que se crea dándole un nombre en contexto de lo que se trata el proyecto y dentro de esa carpeta.

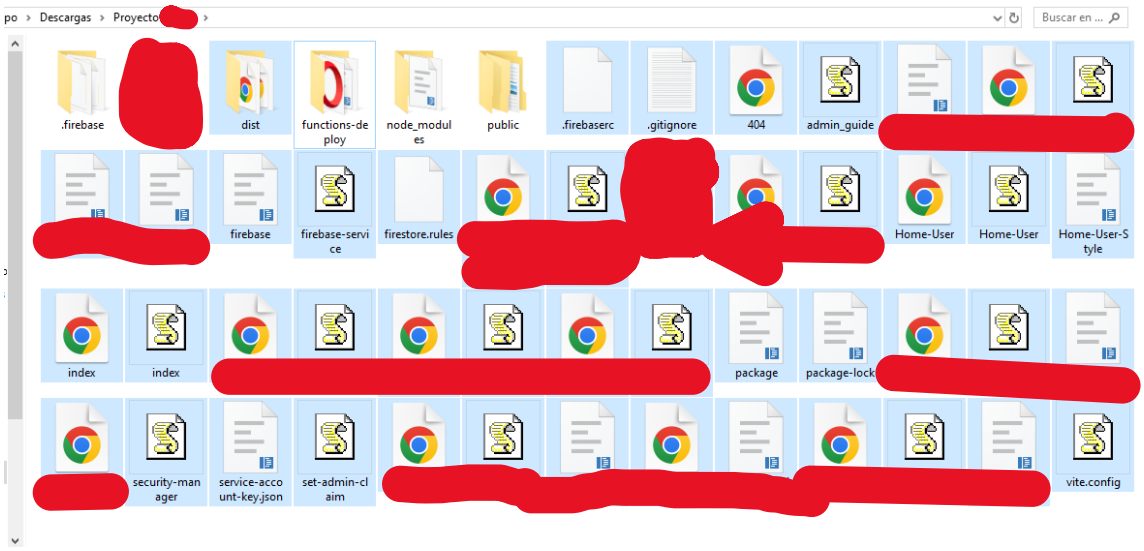
Se crean más carpetas que son subcarpetas que dividen por clasificación o categorías la información de lo que va a manejar la página web como tal.

Y como resultado final dentro de esa dentro de esas carpetas está la carpeta que almacena y alberga todo tu proyecto en html, css, y JavaScript, (SI ES QUE ESTAN MODULARIZADOS OBVIO). ***TODOS*** los documentos que se encuentran dentro de esa carpeta tú debes Seleccionarlos (***TODOS***, y cortarlos, o sea, moverlos) a la carpeta raíz que es la primera carpeta que tú creaste.

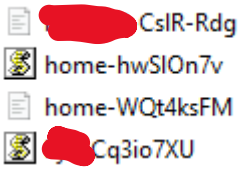


Y tienes que sacar todos esos documentos y soltarlos libres todos dentro de la carpeta raíz, porque luego la herramienta (Vita) hace una copia y escaneo de todos esos documentos.

(*Aquí: ya están todos sueltos, en la carpeta raíz.*)



Cuando más adelante yo te enseñe cómo seleccionar y redirigir el escaneo, específicamente hacia los documentos de tu interés y (Vite); Copia todos los documentos y crea una nueva carpeta llamada (dist) y los anexa, todos los documentos dentro de la carpeta dist, esos documentos vienen con el nombre ofuscado y vuelto una sopa de letras Para irle dando la primera base de forma al código de ofuscaje completo que va a tener todo el proyecto al final.
Ejemplo visual de como te quedaran los archivos escaneados y copiados:



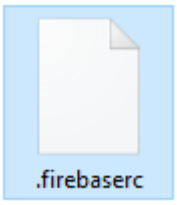
Punto Crítico (Lección Aprendida): Si tienes una carpeta de iconos llamada icons, antes de que tengas problemas futuros con la ruptura de las imágenes que se llaman y se comunican con todo tu html debes moverlas a la ***CARPETA*** de public *osea debes mover la carpeta icons a public*).

- *Estructura Correcta:* public/icons/imagen.png
- *Referencia en Código:* <img src="public/icons/imagen.png"> (Vite no asume que / es la carpeta public) **asi que debes tener la ruta completa en el archivo html** normal que se trabaja en el editor de codigo para que las imágenes puedan ser leídas y subidas por el deploy de Firebase al hacer un only hosting.

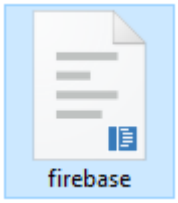
Archivos Requeridos de Firebase:

Un proyecto listo para Firebase debe tener estos archivos en la raíz:

- . firebaserc

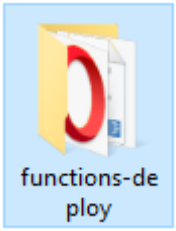


- firebase. json



- firestore.rules / database.rules. json (Si aplica) en el caso de mi proyecto si tengo estos 2 documentos asi que posiblemente también los tengas, por si tienes dudas de en qué lugar deben ir estos documentos siempre van en la carpeta raíz de tu proyecto.

- Carpeta functions/ (Si utilizas Cloud Functions)



---

3. PASO 2: Instalación del Entorno de Construcción (Node.js).

Para que nuestro motor pueda compilar, ofuscar y Minificar el código, es obligatorio tener instalado Node.js en la computadora. Si el usuario ya lo tiene instalado, puede saltar al paso de inicialización.

2.1 Descarga e Instalación de Node.js

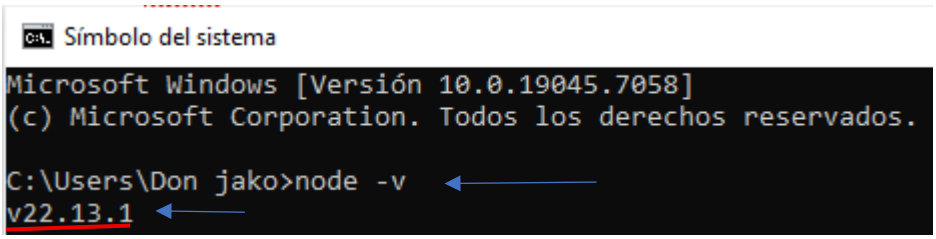
1. Dirígete a la página oficial: nodejs.org
2. Descarga la versión recomendada LTS (Long Term Support), ya que es la más estable.

3. Ejecuta el instalador descargado y sigue los pasos predeterminados del sistema (Siguiente > Siguiente > Instalar).

4. Abre una nueva terminal en tu editor de código (es importante que sea una terminal nueva para que reconozca los cambios) y verifica que la instalación fue exitosa ejecutando estos comandos uno por uno:

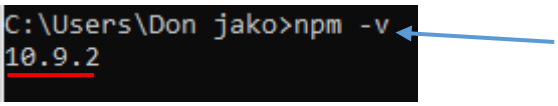
En la Terminal CMD: (*Ejecuta los siguientes comandos****)***

- ✓ node -v: (este comando sirve para que version del programa tenemos, instalada.)



(*Si observas el resultado del primer comando fíjate que en este momento La version de node que estoy usando es la v22.13.1, y un resultado similar Es el que debe salirte en la terminal.*

- ✓ ppm -v: (este comando sirve para que version del programa tenemos, instalada.)



- ✓ *(El sistema debe devolverte los números de las versiones instaladas, por ejemplo: v20.11.0)*. FIN.

A Continuación:

Al iniciar en una carpeta nueva o desde cero (***osea esta es la carpeta del proyecto que ya tenemos creada***), debemos indicarle a Node.js qué herramientas necesitamos, todo lo que realizaremos a continuación se realizara dentro de la terminal del nuestro editor de codigo de preferencia o en el que estemos trabajando en ese momento en este caso estaré realizando las instalaciones en el editor de (***Visual Studio Code***).

Instalación de dependencias y configuración para el despliegue.

Comandos de Terminal para Inicializar:

1. Crear el archivo package. json por defecto:
Debemos abrir la terminal de nuestro editor puedes utilizar (Ctrl + j) o buscar la opcion para desplegar la terminal, *en su editor y una vez esté abierta ejecutaremos el siguiente comando.*

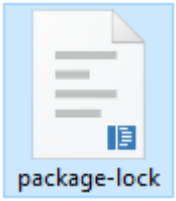
1. **(npm init -y):** (Inicializa el entorno de Node.js y genera automáticamente el archivo de configuración base del proyecto package. json**). y debe dar un resultado como el siguiente:**

```
Problemas   Salida   Consola de depuración   Terminal   Puertos                    powershell

PS C:\Users\Don jako\Downloads\Proyecto [redacted] npm init -y
Wrote to C:\Users\Don jako\Downloads\Proyecto [redacted]\package.json:

{
  "dependencies": {
    "firebase-admin": "^13.6.1",
    "flag-icons": "^7.5.0"
  },
  "name": "proyecto-[redacted]",
  "version": "1.0.0",
  "main": "admin_guide.js",
```

Y el Resultado en el explorador de archivos te va a dejar un documento asi:



Si abres este archivo en tu editor dentro de el te debe quedar un codigo que En el caso de este proyecto tiene dentro una cantidad de 4991 líneas de Configuración de codigo (OJO) quizás esto pueda variar en base a la complejidad de tu proyecto o si la hoja de configuración cambia en función de la herramienta que se está utilizando en el futuro.

2. **npm install vite vite-plugin-javascript-obfuscator vite-plugin-html terser --save-dev:** (Descarga e instala en un solo paso el motor de construcción, los plugins de ofuscación de JS y HTML, y el minificador Terser**). y debe dar un resultado como el siguiente:**

(Primera descarga**).**

```
Problemas   Salida   Consola de depuración   Terminal   Puertos                    powershell

PS C:\Users\Don jako\Downloads\Proyecto [redacted] > npm install terser --save-dev

added 11 packages, and audited 176 packages in 8s

19 packages are looking for funding
  run `npm fund` for details

9 vulnerabilities (8 low, 1 high)

To address issues that do not require attention, run:
  npm audit fix
```

(Como podrás observar el paquete al descargarlo te dice que tiene 9 vulnerabilidades y 1 vulnerabilidad critica, (tranquilo esas vulnerabilidades no son de tu proyecto, simplemente son capas de seguridad de la herramienta que están desactualizadas y se deben actualizar) y eso lo haremos más adelante**).**

(*Si observas como yo tengo escrito el comando a la hora de ejecutarlo en la terminal, podrás ver que le falta una parte por completar, pero eso no importa colócalo completo tal y como esta escrito en esta misma instrucción #2 que es la que está de título OK? Bien*).

(Segunda descarga).

```
PS C:\Users\Don jako\Downloads\Proyecto [redacted] > npm install vite-plugin-html --save-dev

added 59 packages, and audited 356 packages in 28s

76 packages are looking for funding
  run `npm fund` for details

8 low severity vulnerabilities

To address all issues (including breaking changes), run:
```

3. **npm audit fix:** (Analiza y corrige automáticamente las vulnerabilidades menores o dependencias desactualizadas generadas tras la instalación de los paquetes). **y debe dar un resultado como el siguiente:**

   (En 1ra etapa de la auto ejecución y descarga):

```
PS C:\Users\Don jako\Downloads\Proyecto [redacted] > npm audit fix
npm warn deprecated node-domexception@1.0.0: Use your platform's native DOMException instead

added 12 packages, changed 3 packages, and audited 188 packages in 15s

24 packages are looking for funding
  run `npm fund` for details

# npm audit report
```

   (2da y última etapa de la auto ejecución y descarga final):

```
        Depends on vulnerable versions of teeny-request
        node_modules/retry-request
          google-gax  >=4.0.5-experimental
          Depends on vulnerable versions of retry-request
          node_modules/google-gax
            @google-cloud/firestore  7.5.0-pre.0 || >=7.6.0
            Depends on vulnerable versions of google-gax
            node_modules/@google-cloud/firestore

  8 low severity vulnerabilities
```

---

4. PASO 3: Evolución de la Configuración de Vite (vite.config.js)

Este es el cerebro de la operación. A lo largo del proyecto, este archivo evolucionó para aumentar la seguridad. A continuación, se presentan las versiones genéricas para la documentación.

Versión A: Configuración Estándar (Estilo Google - Rápida)

Uso: Ofuscación básica, renombrado de variables a letras simples (a, b, c), alta compatibilidad.

JavaScript

```javascript
// vite.config.js - VERSIÓN A
import { defineConfig } from 'vite';
import javascriptObfuscator from 'vite-plugin-javascript-obfuscator';
import { createHtmlPlugin } from 'vite-plugin-html';
import { resolve } from 'path';

export default defineConfig({
  plugins: [
    // Minificación de HTML básica
    createHtmlPlugin({ minify: true }),

    // Ofuscador Estilo Google (Mangling)
    javascriptObfuscator({
      compact: true,
      controlFlowFlattening: false, // Apagado para evitar conflictos iniciales
      identifierNamesGenerator: 'mangled', // Usa a, b, c
      stringArray: false, // No encripta textos
    }),
  ],
  build: {
    target: 'esnext',
    minify: 'terser', // Compresión estándar
    rollupOptions: {
      input: {
        // Lista genérica de documentos
        main: resolve(__dirname, 'index.html'),
        login: resolve(__dirname, 'login.html'),
        panel: resolve(__dirname, 'panel.html'),
      }
    }
  }
});
```

Versión B: Configuración de Alta Seguridad y Línea Infinita (RECOMENDADA)

Uso: Máxima protección. Encripta textos, usa nombres hexadecimales, desordena la lógica, protege contra depuración (F12) y aplasta todo en una sola línea de texto.

JavaScript

```javascript
// vite.config.js - VERSIÓN B (Máxima Seguridad)
import { defineConfig } from 'vite';
import javascriptObfuscator from 'vite-plugin-javascript-obfuscator';
```

```
import { createHtmlPlugin } from 'vite-plugin-html';
import { resolve } from 'path';

export default defineConfig({
  plugins: [
    // 1. MINIFICACIÓN AGRESIVA DE HTML (Sopa de letras)
    createHtmlPlugin({
      minify: {
        collapseWhitespace: true, // Elimina espacios y saltos de línea
        removeComments: true, // Borra comentarios
        minifyCSS: true, // Comprime CSS interno
        minifyJS: true, // Comprime JS interno
        removeRedundantAttributes: true,
        useShortDoctype: true,
      }
    }),


    // 2. OFUSCACIÓN AGRESIVA DE JS
    javascriptObfuscator({
      compact: true, // Aplasta el JS

      // Encriptación de Textos (Strings)
      stringArray: true,
      stringArrayEncoding: ['base64', 'rc4'], // Doble encriptación ilegible
      stringArrayThreshold: 1, // Encriptar el 100% de los textos

      // Ofuscación de Nombres
      identifierNamesGenerator: 'hexadecimal', // Usa _0x1a2b3c
      renameGlobals: true, // Renombra variables globales

      // Protección y Desorden Lógico
      debugProtection: true, // Congela la página si abren F12
      debugProtectionInterval: 4000,
      disableConsoleOutput: true, // Elimina console.logs
      controlFlowFlattening: true, // Desordena el flujo del código
      controlFlowFlatteningThreshold: 0.5,

      // Nombres reservados para no romper APIs (Firebase, etc.)
      reservedNames: ['^fb', '^firebase', '^Vite', 'require']
    }),
  ],
  build: {
    target: 'esnext',

    // 3. MINIFICACIÓN EXTREMA FINAL (Línea Infinita)
    minify: 'terser',
    terserOptions: {
      format: {
        comments: false, // Garantiza cero comentarios
      },
      compress: {
        drop_console: true, // Elimina console.logs residuales
        drop_debugger: true
```

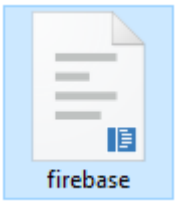
```
      }
    },
    rollupOptions: {
      input: {
        // IMPORTANTE: Listar TODOS los HTML del proyecto aquí
        main: resolve(__dirname, 'index.html'),
        login: resolve(__dirname, 'login.html'),
        panel: resolve(__dirname, 'panel.html'),
        // ... añadir el resto de documentos
      }
    }
  }
});
```

---

5. Configuración del Motor de Hosting (Firebase).

Antes de construir el proyecto, debes indicarle a **Firebase** qué carpeta debe subir a internet. Por defecto, **Firebase** busca una carpeta llamada **public**, pero Vite generará el código ofuscado en una carpeta llamada **dist**.

1. Abre el archivo **firebase.json** ubicado en la raíz de tu proyecto.



2. Localiza la sección "hosting".

3. Cambia el valor "**public**": que debe ser actualmente "**public**" por : "**dist**".

6. Construcción, Ofuscación y Minificación Total (The Build).

npx vite build : (Lee el código fuente original y ejecuta tres procesos simultáneos:

1. Aplica la ofuscación extrema al JavaScript.

2. Comprime el HTML.

3. **Minifica automáticamente todos los archivos CSS**, eliminando espacios y comentarios para convertirlos en una sola línea de código ["sopa de letras"]. Finalmente, empaqueta todo el resultado en la nueva carpeta dist**).**

y debe dar un resultado como el siguiente: (A Continuación, veras la compresión y Minificación de todos los archivos de tu proyecto Transformados a nombres extraños esto es totalmente normal.**)**

1ra parte.

```
PS C:\Users\Don jako\Downloads\Proyecto [redacted] > npx vite build
vite v8.0.2 building client environment for production...
✓ 39 modules transformed.
computing gzip size...
dist/404.html                                          1.89 kB | gzip:  0.91 kB
dist/Formulario-de-recuperacion-de-contraseña.html     2.81 kB | gzip:  1.17 kB
dist/nuevo-password.html                               4.21 kB | gzip:  1.36 kB
dist/singnup.html                                      8.61 kB | gzip:  2.72 kB
dist/politica-de-privacidad.html                       9.29 kB | gzip:  3.39 kB
```

2da parte.

```
dist/assets/TyC style and PfP style-B3a_DOcb.css       4.58 kB | gzip:  1.31 kB
dist/assets/kyc-DzBpqAK8.css                          10.26 kB | gzip:  2.70 kB
dist/assets/historial-CslR-Rdg.css                    12.10 kB | gzip:  3.18 kB
dist/assets/seguridad-v765RWR3.css                    13.35 kB | gzip:  3.44 kB
dist/assets/css-del-inicio-de-sesion-DIo1v7M1.css     13.47 kB | gzip:  3.50 kB
dist/assets/style-my-account-xZw8rJGh.css             20.84 kB | gzip:  4.87 kB
dist/assets/home-WQt4ksFM.css                         29.83 kB | gzip:  6.76 kB
dist/assets/admin-BeKXYrPg.css                        41.94 kB | gzip:  8.60 kB
dist/assets/modulepreload-polyfill-DUujy55Z.js         1.40 kB | gzip:  0.76 kB
dist/assets/chunk-DWNuXI3u.js                          1.62 kB | gzip:  0.94 kB
dist/assets/security-manager-QVppLGY0.js               1.72 kB | gzip:  1.07 kB
```

Resultado final y muestra de la carpeta en el explorador de archivos.

Al final cuando la herramienta termine de comprimir y ofuscar todo el codigo

te dejara un resultado asi en la terminal:

```
 WARN [PLUGIN_TIMINGS] Warning: Your build spent significant time in plugins. Here is a breakdown:
   - vite:build-html (56%)
   - vite-plugin-javascript-obfuscator (34%)
See https://rolldown.rs/options/checks#plugintimings for more details.


✓ built in 4.92s
```

En el explorador de archivos se creare automáticamente en segundo plano

Una carpeta como la siguiente:

Error Común a tomar en cuenta: (Tomar en cuenta solo si llega a suceder.**)**
Si la terminal lanza un error amarillo o rojo como Unable to parse HTML, significa que tu HTML tiene un error de sintaxis (falta una comilla en un atributo, un style= mal cerrado, etc.). Debes corregirlo en el archivo raíz y volver a correr el build.

```
PS C:\Users\Don jako\Downloads\Proyecto [redacted] > npx vite build
vite v8.0.2 building client environment for production...
transforming (1) rolldown\runtime.js
Unable to parse HTML; parse5 error code unexpected-character-in-unquoted-attribute-value
 at C:/Users/Don jako/Downloads/Proyecto [redacted] /Home-User.html:625:88
632 |                                <!-- [redacted] -->
633 |                                <div class="option-card" onclick="select[redacted]('[redacted]', '[redacted]')">
634 |                                    <img src="public/icons_cripto/toncoin-ton-logo.svg" alt="[redacted]" style="width:32px; h
eig...
    |                ^
635 |                                    <div style="flex: 1;">
```

los archivos HTML ahora están aquí, y una carpeta assets contiene los JS y CSS con nombres raros (ej. main-Bg1H2j3k.js).

> Descargas > Proyecto > dist > assets

Nombre	Fecha de modificación	Tipo	Tamaño
admin-BeKXYrPg	23/3/2026 2:11 p. m.	Archivo CSS	41 KB
admin-Ygicc_87	23/3/2026 2:11 p. m.	Archivo JavaScript	53 KB
i0wVzkC	23/3/2026 2:11 p. m.	Microsoft Edge H...	6 KB
chunk-U8z2eGLg	23/3/2026 2:11 p. m.	Archivo JavaScript	2 KB
css-del-inicio-de-sesion-Dlo1v7M1	23/3/2026 2:11 p. m.	Archivo CSS	14 KB
-Cy8MiZHN	23/3/2026 2:11 p. m.	Microsoft Edge H...	56 KB
firebase-service-D3We7Qat	23/3/2026 2:11 p. m.	Archivo JavaScript	10 KB
historial-Cc7Q4gMU	23/3/2026 2:11 p. m.	Archivo JavaScript	21 KB
historial-CsIR-Rdg	23/3/2026 2:11 p. m.	Archivo CSS	12 KB
home-hwSIOn7v	23/3/2026 2:11 p. m.	Archivo JavaScript	66 KB
home-WQt4ksFM	23/3/2026 2:11 p. m.	Archivo CSS	30 KB
kyc-Cq3io7XU	23/3/2026 2:11 p. m.	Archivo JavaScript	13 KB
kyc-DzBpqAK8	23/3/2026 2:11 p. m.	Archivo CSS	11 KB
main-Co-cAWBO	23/3/2026 2:11 p. m.	Archivo JavaScript	8 KB
micuenta-DNdleBY2	23/3/2026 2:11 p. m.	Archivo JavaScript	9 KB
modulepreload-polyfill-CtzYEavj	23/3/2026 2:11 p. m.	Archivo JavaScript	2 KB
-D_5Lw0uj	23/3/2026 2:11 p. m.	Archivo JavaScript	14 KB
password-Ct8hmAKV	23/3/2026 2:11 p. m.	Archivo JavaScript	4 KB
recuperacion-DVWWmAYK	23/3/2026 2:11 p. m.	Archivo JavaScript	3 KB
security-manager-FH4CA_Db	23/3/2026 2:11 p. m.	Archivo JavaScript	2 KB
seguridad-BEx_Ibhj	23/3/2026 2:11 p. m.	Archivo JavaScript	14 KB
seguridad-v765RWR3	23/3/2026 2:11 p. m.	Archivo CSS	14 KB
signup-wJOYtldO	23/3/2026 2:11 p. m.	Archivo JavaScript	7 KB
style-my-account-xZw8rJGh	23/3/2026 2:11 p. m.	Archivo CSS	21 KB

---

7. PASO 4: Despliegue y Verificación Final

Último paso: decirle a Firebase que suba la carpeta `dist` (la ofuscada) y no la carpeta raíz.

Comando de Despliegue:

1. **firebase deploy --only hosting:** (Y te debe dar un resultado como el siguiente.)

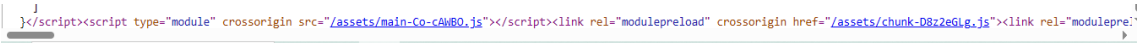
```
PS C:\Users\Don jako\Downloads\Proyecto > firebase deploy --only hosting
(node:8576) [DEP0040] DeprecationWarning: The `punycode` module is deprecated. Please use a userland alternative instead.
(Use `node --trace-deprecation ...` to show where the warning was created)

=== Deploying to '...'

i  deploying hosting
i  hosting[crypto-imperial]: beginning deploy...
i  hosting[crypto-imperial]: found 53 files in dist
+  hosting[crypto-imperial]: file upload complete
i  hosting[crypto-imperial]: finalizing version...
```

Verificación de la "Línea Infinita":

Entra a tu página web ya subida, haz clic derecho y selecciona **"Ver código fuente de la página"**. En el caso de este este proyecto obtuve una compresión del HTML llevando toda sintaxis a una sola línea de 50,000 MIL Caracteres en una sola línea infinita. Ese debe ser tu resultado final espera dependiendo de que configuración hayas elegido.



FIN DE LA DOCUMENTACIÓN

GRACIAS POR LLEGAR HASTA AQUÍ ESTE DOCUMENTO FUE CRADO POR.

(VAR.P-R233)

.